

Bill Bot — Source-Code Risk Acceptance Memo

Response to findings raised in the Source Code Scanning Report dated 22 May 2026.

Application	Bill Bot
Source File Reviewed	Final_pdf_extractor4.py
Scan Report Reference	Source Code Scanning Report, 22 May 2026
Date	29 May 2026
Author	Huzaifa Manasawala

Part A — Findings Remediated in Code

The following findings have been addressed through direct modification of the application source. A brief description of each change and the affected location in `Final_pdf_extractor4.py` is given below.

#	Title	Severity	What changed
1	Insecure Temporary CSV File Creation	Medium	The application no longer writes a temporary CSV copy of the input template. The Excel template is read directly with type-safe handling for Artiva account identifier columns, so no extra copy of PII persists on disk. (lines ~350–378)
2	Information Disclosure Through Verbose Exception Logging	Medium	The PDF text-extraction routine no longer embeds the full filesystem path or the Python traceback in user-facing error messages. The traceback is routed to the internal log file only; the user-visible message carries the filename alone. (lines ~60–82)
6	Dependency Security Risk — Third-Party Libraries Without Version Pinning	Medium	A <code>requirements.txt</code> file has been added with exact version pins for every direct runtime dependency. The dependency set is scanned for known CVEs using <code>pip-audit</code> on a quarterly cadence and before every release build.
7	Suppression of Security-Relevant Warnings	Low	The pypdf warning suppression at module load has been narrowed from "all warnings" to "informational user warnings only." Deprecation, future-API, and resource-leak warnings from pypdf are no longer hidden. (lines ~17–20)
10	Excel Formula Injection Risk	High	A formula-neutralization step prefixes a single-quote literal escape onto any cell value beginning with <code>=</code> , <code>+</code> , <code>-</code> , <code>@</code> , tab, or carriage return before either output Excel file is written. This applies to all text columns of both <code>Extracted_Output.xlsx</code> and <code>Processing_Log.xlsx</code> . (lines ~1083–1086 and ~1102–1105)
12	Missing Integrity Validation for Imported PDFs	Low	The PDF text-extraction routine validates the <code>%PDF-</code> magic header on every input file before invoking the parser. Files that

		do not begin with that signature are rejected with a clear error and excluded from processing. (lines ~60–64)
--	--	---

Implementation Note — Localized Warning Suppression Introduced During Risk #1 Remediation. During the Risk #1 remediation, a narrowly-scoped Python warning suppression was added around the single call that reads the input Excel template. It silences one specific informational pandas advisory that appears when a converter and a default type are both applied to the same column — which is the case for the three Artiva account-identifier columns handled by the converter. The suppression is active only during that one read call and cannot mask deprecation, future-API, resource-leak, or security warnings from any other part of the application; normal warning behaviour resumes automatically as soon as the read completes. This is structurally different from the original Risk #7 issue — a global, lifetime-long suppression of every pypdf warning — which was independently remediated by narrowing the filter to informational user warnings only. The localized block here is the recommended Python pattern for silencing a single known-safe advisory and does not reintroduce the security concern cited in Risk #7. (line ~354)

Part B — Findings Risk-Accepted

The following findings have been formally accepted as residual risk on the basis of the application's deployment model (single-user, packaged Windows executable distributed to PCI-zone workstations; no network or upload interface; trusted upstream input source) and the surrounding PCI DSS environmental controls. A finding-specific justification is provided for each.

Risk #3 — Sensitive Data Leakage in Logs (Medium)

Status: Risk Accepted.

The values logged (Artiva CRM account numbers, PDF filenames, and processing states) are internal operational identifiers used to reconcile processing failures against the input template. Artiva account numbers function as CRM record keys, not financial-instrument identifiers. Log artifacts (`bill_processing.log` , `Processing_Log.xlsx`) are written only to the authorized operator's own user-profile folder on a PCI-zone workstation, with no network log shipping and no multi-user filesystem access path. Masking Artiva account numbers would defeat the diagnostic purpose of the logs while adding no security benefit beyond what the environment enforces; residual risk is bounded to compromise of the workstation itself, which lies outside the application's threat boundary.

Risk #4 — Unvalidated File Path Usage (Medium)

Status: Risk Accepted.

Filenames consumed by the application are sourced exclusively from `os.listdir()` against a hardcoded input directory (`%USERPROFILE%\Desktop\Bills`); the application accepts no user-supplied paths, no command-line path arguments, and no network input, so file selection is entirely determined by what the authorized operator places in that location. All filesystem sinks (`open` , `pdfplumber.open` , `PdfWriter.append` , `shutil.copyfile`) consume paths built via `os.path.join(folder_path, basename)` where `basename` comes from `os.listdir` and `folder_path` is fixed, leaving no concatenation surface for an externally-controlled traversal sequence. For the cited path-traversal threat to be realized, an attacker would first require authenticated local write access to the operator's workstation — at which point traversal through this code is the least of the available attack vectors. PCI-zone endpoint hardening and access control govern who can introduce files into the input directory, so residual risk attributable to the application is none.

Risk #5 — Unsafe PDF Processing of Untrusted Files (Medium)

Status: Risk Accepted, with compensating controls.

This finding assumes a web service that accepts PDF uploads from anonymous users — that is not how this application works. It is distributed as a Windows executable that runs only on PCI-zone workstations, has no network or upload interface, and processes only PDFs the authorized operator has personally downloaded from authenticated client/telco portals (AT&T, Comcast, DirecTV), which are well-formed documents produced by enterprise billing systems rather than user-supplied uploads. The Denial-of-Service concern does not apply because the tool is a single-user batch job, not a service — an interrupted run is simply re-started. Compensating controls are in place: the application checks the `%PDF-` file header before parsing each file (so non-PDFs and corrupted files are rejected early), pins all PDF-parsing libraries to exact versions in `requirements.txt`, and scans the dependency set for known CVEs using `pip-audit` on a quarterly cadence and before every release build. Sandboxing, file-size limits, and cryptographic file-integrity checks were not added — sandboxing is disproportionate for a single-user offline tool, fixed size caps would block legitimate large client bills, and telcos do not publish file hashes to validate against. Residual risk is limited to an undisclosed CVE in the parsing libraries being triggered by a malicious-but-well-formed PDF, monitored through the dependency-scanning cycle.

Risk #8 — Potential Denial of Service Through Unbounded File Processing (Medium)

Status: Risk Accepted.

This application is a single-user batch tool, not a long-running service, so the cited Denial-of-Service impact translates only to the operator's own batch taking longer or needing to be re-run — an inconvenience, not a security incident affecting any other user or shared system. The set of input files is limited to what the authorized operator manually downloads and places in the input folder; there is no automated feed, upload endpoint, or shared-folder watcher through which an outside party could flood the tool with files. The PCI-zone workstation already provides OS-level resource monitoring, and the application shows a live progress bar and per-account log entries, so any unexpectedly large batch is immediately visible to the operator before any resource exhaustion. Residual security risk is negligible and the operational risk of long-running batches is accepted as recoverable.

Risk #9 — Lack of Input Validation on Excel/CSV Data (Medium)

Status: Risk Accepted — addressed transitively by Risk #10's output-layer mitigation.

This finding describes the same threat as Risk #10 — a spreadsheet program evaluating cell content as a formula — but at the input-read step instead of the output-write step. When the application reads the input template, pandas does not evaluate cell contents as formulas, so a hostile leading character in any input cell has no effect on the running process. The actual point of exploitation is when extracted data is written into the output Excel files, and the mitigation implemented for Risk #10 already prefixes a single-quote escape onto any value beginning with a formula-trigger character before either output file is written. This covers every hostile value regardless of whether it originated in the input template, the PDF text, or generated log messages. Adding redundant input-side sanitization would not increase coverage and could accidentally corrupt legitimate negative-amount values before numeric processing. Residual risk is zero — the threat is fully handled at the output layer.

Risk #11 — Insufficient Access Control Around Output Files (Medium)

Status: Risk Accepted.

The fix suggested by the scanner (setting file permissions via Linux-style mode bits) is a Linux-only mechanism and does nothing on Windows, so applying it would not actually change any access control. Output folders are created inside the authorized operator's own user profile (under `%USERPROFILE%\Desktop`), and Windows already restricts access to that area to the operator's account and local administrators — the same protection that already applies to every other file in the operator's profile, including the input PDFs and the input template. The workstation itself is governed by PCI-zone controls (authenticated sign-in, full-disk encryption, endpoint protection, audit logging), and the operator is both the creator and the intended reader of the output files, so the "unauthorized local users" the scanner contemplates do not exist in this single-operator setup. Adding additional Windows ACL hardening on top of the existing default protection would not meaningfully reduce risk and would introduce maintenance overhead disproportionate to any benefit. Residual risk attributable to the application is none — all meaningful access controls are environmental.

Risk #13 — Potential Exposure of Internal Business Logic in Comments and Logs (Low)

Status: Risk Accepted.

The application is distributed as a compiled Windows executable, not as readable source code, so internal comments and operational log messages are not exposed outside the development environment. The categories of information the scanner is concerned about (validation rules, version-merging logic, account-handling behavior) are inherent to the tool's purpose and would be observable to any party with normal use of the tool through a few minutes of input/output testing — concealing comments would not impede reconnaissance, only the developer's ability to safely maintain the code. The source repository is held in an access-controlled internal store outside the deployment boundary, and the workstation is governed by PCI-zone environmental controls. Residual risk of business-logic disclosure is negligible.